INTERNET ARCHIVE
WaybackMachine
http://www.ibm.com/developerworks/linux/library/l-gas-nasm/index.html Go
NOV OCT DEC
03
2012 2013
42 captures
17 Apr 2011 - 2 Apr 2019
About this capture

Linux assemblers: A comparison of GAS and NASM

A side-by-side look at GNU Assembler (GAS) and Netwide Assembler (NASM)

Ram Narayan, Software Engineer, IBM

Summary: This article explains some of the more important syntactic and semantic differences between two of the most popular assemblers for Linux®, GNU Assembler (GAS) and Netwide Assembler (NASM), including differences in basic syntax, variables and memory access, macro handling, functions and external routines, stack handling, and techniques for easily repeating blocks of code.

Date: 17 Oct 2007
Also available in: Russian Japanese

Activity: 11388 views
Comments: (View | Add comment - Sign in)

Rate this article

Introduction

Unlike other languages, **assembly programming** involves understanding the processor architecture of the machine that is being programmed. Assembly programs are not at all portable and are often cumbersome to maintain and understand, and can often contain a large number of lines of code. But with these limitations comes the advantage of speed and size of the runtime binary that executes on that machine.

Though much information is already available on assembly level programming on Linux, this article aims to more specifically show the differences between syntaxes in a way that will help you more easily convert from one flavor of assembly to the another. The article evolved from my own quest to improve at this conversion.

This article uses a series of program examples. Each program illustrates some feature and is followed by a discussion and comparison of the syntaxes. Although it's not possible to cover every difference that exists between NASM and GAS, I do try to cover the main points and provide a foundation for further investigation. And for those already familiar with both NASM and GAS, you might still find something useful here, such as macros.

This article assumes you have at least a basic understanding of assembly terminology and have programmed with an assembler using Intel® syntax, perhaps using NASM on Linux or Windows. This article does not teach how to type code into an editor or how to assemble and link (but see the sidebar for a quick refresher). You should be familiar with the Linux operating system (any Linux distribution will do; I used Red Hat and Slackware) and basic GNU tools such as gcc and ld, and you should be programming on an x86 machine.

Now I'll describe what this article does and does not cover.

Building the examples

Assembling:
GAS:
```
as -o program.o program.s
```

NASM:
```
nasm -f elf -o program.o program.asm
```

Linking (common to both kinds of assembler):
```
ld -o program program.o
```

Linking when an external C library is to be used:
`ld --dynamic-linker /lib/ld-linux.so.2 -lc -o program program.o`

This article covers:

- Basic syntactical differences between NASM and GAS
- Common assembly level constructs such as variables, loops, labels, and macros
- A bit about calling external C routines and using functions
- Assembly mnemonic differences and usage
- Memory addressing methods

This article does not cover:

- The processor instruction set
- Various forms of macros and other constructs particular to an assembler
- Assembler directives peculiar to either NASM or GAS
- Features that are not commonly used or are found only in one assembler but not in the other

For more information, refer to the official assembler manuals (see Resources for links), as those are the most complete sources of information.

Basic structure

Listing 1 shows a very simple program that simply exits with an exit code of 2. This little program describes the basic structure of an assembly program for both GAS and NASM.

Listing 1. A program that exits with an exit code of 2

Line	NASM	GAS
001	`; Text segment begins`	`# Text segment begins`
002	`section .text`	`.section .text`
003		
004	`global _start`	`.globl _start`
005		
006	`; Program entry point`	`# Program entry point`
007	`_start:`	`_start:`
008		
009	`; Put the code number for system call`	`# Put the code number for system call`
010	`mov eax, 1`	`movl $1, %eax`
011		
012	`; Return value`	`/* Return value */`
013	`mov ebx, 2`	`movl $2, %ebx`
014		
015	`; Call the OS`	`# Call the OS`
016	`int 80h`	`int $0x80`

Now for a bit of explanation.

One of the biggest differences between NASM and GAS is the syntax. GAS uses the AT&T syntax, a relatively archaic syntax that is specific to GAS and some older assemblers, whereas NASM uses the Intel syntax, supported by a majority of assemblers such as TASM and MASM. (Modern versions of GAS do support a directive called `.intel_syntax`, which allows the use of Intel syntax with GAS.)

The following are some of the major differences summarized from the GAS manual:

- AT&T and Intel syntax use the opposite order for source and destination operands. For example:
  - Intel: `mov eax, 4`
  - AT&T: `movl $4, %eax`
- In AT&T syntax, immediate operands are preceded by `$`; in Intel syntax, immediate operands are not. For example:
  - Intel: `push 4`
  - AT&T: `pushl $4`
- In AT&T syntax, register operands are preceded by `%`; in Intel syntax, they are not.
- In AT&T syntax, the size of memory operands is determined from the last character of the opcode name. Opcode suffixes of `b`, `w`, and `l` specify byte (8-bit), word (16-bit), and long (32-bit) memory references. Intel syntax accomplishes this by prefixing memory operands

(not the opcodes themselves) with `byte ptr`, `word ptr`, and `dword ptr`. Thus:
  - Intel: `mov al, byte ptr foo`
  - AT&T: `movb foo, %al`
- Immediate form long jumps and calls are `lcall/ljmp $section, $offset` in AT&T syntax; the Intel syntax is `call/jmp far section:offset`. The far return instruction is `lret $stack-adjust` in AT&T syntax, whereas Intel uses `ret far stack-adjust`.

In both the assemblers, the names of registers remain the same, but the syntax for using them is different as is the syntax for addressing modes. In addition, assembler directives in GAS begin with a ".", but not in NASM.

The `.text` section is where the processor begins code execution. The `global` (also `.globl` or `.global` in GAS) keyword is used to make a symbol visible to the linker and available to other linking object modules. On the NASM side of Listing 1, `global _start` marks the symbol `_start` as a visible identifier so the linker knows where to jump into the program and begin execution. As with NASM, GAS looks for this `_start` label as the default entry point of a program. A label always ends with a colon in both GAS and NASM.

Interrupts are a way to inform the OS that its services are required. The `int` instruction in line 16 does this job in our program. Both GAS and NASM use the same mnemonic for interrupts. GAS uses the `0x` prefix to specify a hex number, whereas NASM uses the `h` suffix. Because immediate operands are prefixed with `$` in GAS, 80 hex is `$0x80`.

`int $0x80` (or `80h` in NASM) is used to invoke Linux and request a service. The service code is present in the EAX register. A value of 1 (for the Linux exit system call) is stored in EAX to request that the program exit. Register EBX contains the exit code (2, in our case), a number that is returned to the OS. (You can track this number by typing `echo $?` at the command prompt.)

Finally, a word about comments. GAS supports both C style (`/* */`), C++ style (`//`), and shell style (`#`) comments. NASM supports single-line comments that begin with the ";" character.

Variables and accessing memory

This section begins with an example program that finds the largest of three numbers.

Listing 2. A program that finds the maximum of three numbers

Line	NASM	GAS
001	`; Data section begins`	`// Data section begins`
002	`section .data`	`.section .data`
003		
004	`var1 dd 40`	`var1:`
005		`.int 40`
006	`var2 dd 20`	`var2:`
007		`.int 20`
008	`var3 dd 30`	`var3:`
009		`.int 30`
010		
011	`section .text`	`.section .text`
012		
013	`global _start`	`.globl _start`
014		
015	`_start:`	`_start:`
016		
017	`; Move the contents of variables`	`# move the contents of variables`
018	`mov ecx, [var1]`	`movl (var1), %ecx`
019	`cmp ecx, [var2]`	`cmpl (var2), %ecx`
020	`jg check_third_var`	`jg check_third_var`
021	`mov ecx, [var2]`	`movl (var2), %ecx`
022		
023	`check_third_var:`	`check_third_var:`
024	`cmp ecx, [var3]`	`cmpl (var3), %ecx`
025	`jg _exit`	`jg _exit`
026	`mov ecx, [var3]`	`movl (var3), %ecx`
027		
028	`_exit:`	`_exit:`
029	`mov eax, 1`	`movl $1, %eax`
030	`mov ebx, ecx`	`movl %ecx, %ebx`
031	`int 80h`	`int $0x80`

You can see several differences above in the declaration of memory variables. NASM uses the `dd`, `dw`, and `db` directives to declare 32-, 16-, and 8-bit numbers, respectively, whereas GAS uses the `.long`, `.int`, and `.byte` for the same purpose. GAS has other directives too, such as `.ascii`, `.asciz`, and `.string`. In GAS, you declare variables just like other labels (using a colon), but in NASM you simply type a variable name (without the colon) before the memory allocation directive (`dd`, `dw`, etc.), followed by the value of the variable.

Line 18 in Listing 2 illustrates the memory indirect addressing mode. NASM uses square brackets to dereference the value at the address pointed to by a memory location: `[var1]`. GAS uses a circular brace to dereference the same value: `(var1)`. The use of other addressing modes is covered later in this article.

Using macros

Listing 3 illustrates the concepts of this section; it accepts the user's name as input and returns a greeting.

Listing 3. A program to read a string and display a greeting to the user

Line	NASM	GAS
001	`section .data`	`.section .data`
002		
003	`   prompt_str  db   'Enter your name: '`	`   prompt_str:`
004		`      .ascii "Enter Your Name: "`
005	`; $ is the location counter`	`   pstr_end:`
006	`   STR_SIZE  equ  $ - prompt_str`	`      .set STR_SIZE, pstr_end - prompt_str`
007		
008	`   greet_str  db  'Hello '`	`   greet_str:`
009		`      .ascii "Hello "`
010		
011	`   GSTR_SIZE  equ  $ - greet_str`	`   gstr_end:`
012		`      .set GSTR_SIZE, gstr_end - greet_str`
013		
014	`section .bss`	`.section .bss`
015		
016	`; Reserve 32 bytes of memory`	`// Reserve 32 bytes of memory`
017	`   buff  resb  32`	`   .lcomm  buff, 32`
018		
019	`; A macro with two parameters`	`// A macro with two parameters`
020	`; Implements the write system call`	`//  implements the write system call`
021	`   %macro write 2`	`   .macro write str, str_size`
022	`      mov   eax, 4`	`      movl  $4, %eax`
023	`      mov   ebx, 1`	`      movl  $1, %ebx`
024	`      mov   ecx, %1`	`      movl  \str, %ecx`
025	`      mov   edx, %2`	`      movl  \str_size, %edx`
026	`      int   80h`	`      int   $0x80`
027	`   %endmacro`	`   .endm`
028		
029		
030	`; Implements the read system call`	`// Implements the read system call`
031	`   %macro read 2`	`   .macro read buff, buff_size`
032	`      mov   eax, 3`	`      movl  $3, %eax`
033	`      mov   ebx, 0`	`      movl  $0, %ebx`
034	`      mov   ecx, %1`	`      movl  \buff, %ecx`
035	`      mov   edx, %2`	`      movl  \buff_size, %edx`
036	`      int   80h`	`      int   $0x80`
037	`   %endmacro`	`   .endm`
038		
039		
040	`section .text`	`.section .text`
041		
042	`   global _start`	`   .globl _start`
043		
044	`   _start:`	`   _start:`
045	`      write prompt_str, STR_SIZE`	`      write $prompt_str, $STR_SIZE`
046	`      read  buff, 32`	`      read  $buff, $32`
047		
048	`; Read returns the length in eax`	`// Read returns the length in eax`
049	`      push  eax`	`      pushl %eax`
050		
051	`; Print the hello text`	`// Print the hello text`
052	`      write greet_str, GSTR_SIZE`	`      write $greet_str, $GSTR_SIZE`
053		

```
054         pop   edx                                          popl  %edx
055
056    ; edx  = length returned by read                  // edx = length returned by read
057         write buff, edx                                write $buff, %edx
058
059      _exit:                                            _exit:
060         mov   eax, 1                                      movl  $1, %eax
061         mov   ebx, 0                                      movl  $0, %ebx
062         int   80h                                         int   $0x80
```

The heading for this section promises a discussion of macros, and both NASM and GAS certainly support them. But before we get into macros, a few other features are worth comparing.

Listing 3 illustrates the concept of uninitialized memory, defined using the `.bss` section directive (line 14). BSS stands for "block storage segment" (originally, "block started by symbol"), and the memory reserved in the BSS section is initialized to zero during the start of the program. Objects in the BSS section have only a name and a size, and no value. Variables declared in the BSS section don't actually take space, unlike in the data segment.

NASM uses the `resb`, `resw`, and `resd` keywords to allocated byte, word, and dword space in the BSS section. GAS, on the other hand, uses the `.lcomm` keyword to allocate byte-level space. Notice the way the variable name is declared in both versions of the program. In NASM the variable name precedes the `resb` (or `resw` or `resd`) keyword, followed by the amount of space to be reserved, whereas in GAS the variable name follows the `.lcomm` keyword, which is then followed by a comma and then the amount of space to be reserved. This shows the difference:

NASM: `varname resb size`

GAS: `.lcomm varname, size`

Listing 2 also introduces the concept of a location counter (line 6). NASM provides a special variable (the `$` and `$$` variables) to manipulate the location counter. In GAS, there is no method to manipulate the location counter and you have to use labels to calculate the next storage location (data, instruction, etc.).

For example, to calculate the length of a string, you would use the following idiom in NASM:

```
prompt_str db 'Enter your name: '
STR_SIZE equ $ - prompt_str     ; $ is the location counter
```

The `$` gives the current value of the location counter, and subtracting the value of the label (all variable names are labels) from this location counter gives the number of bytes present between the declaration of the label and the current location. The `equ` directive is used to set the value of the variable STR_SIZE to the expression following it. A similar idiom in GAS looks like this:

```
prompt_str:
     .ascii "Enter Your Name: "

pstr_end:
     .set STR_SIZE, pstr_end - prompt_str
```

The end label (`pstr_end`) gives the next location address, and subtracting the starting label address gives the size. Also note the use of `.set` to initialize the value of the variable STR_SIZE to the expression following the comma. A corresponding `.equ` can also be used. There is no alternative to GAS's `set` directive in NASM.

As I mentioned, Listing 3 uses macros (line 21). Different macro techniques exist in NASM and GAS, including single-line macros and macro overloading, but I only deal with the basic type here. A common use of macros in assembly is clarity. Instead of typing the same piece of code again and again, you can create reusable macros that both avoid this repetition and enhance the look and readability of the code by reducing clutter.

NASM users might be familiar with declaring macros using the `%beginmacro` directive and ending them with an `%endmacro` directive. A `%beginmacro` directive is followed by the macro name. After the macro name comes a count, the number of macro arguments the macro is supposed to have. In NASM, macro arguments are numbered sequentially starting with 1. That is, the first argument to a macro is %1, the second is %2, the third is %3, and so on. For example:

```
%beginmacro macroname 2
     mov eax, %1
     mov ebx, %2
```

```
%endmacro
```

This creates a macro with two arguments, the first being `%1` and the second being `%2`. Thus, a call to the above macro would look something like this:

```
macroname 5, 6
```

Macros can also be created without arguments, in which case they don't specify any number.

Now let's take a look at how GAS uses macros. GAS provides the `.macro` and `.endm` directives to create macros. A `.macro` directive is followed by a macro name, which may or may not have arguments. In GAS, macro arguments are given by name. For example:

```
.macro macroname arg1, arg2
     movl \arg1, %eax
     movl \arg2, %ebx
.endm
```

A backslash precedes the name of each argument of the macro when the name is actually used inside a macro. If this is not done, the linker would treat the names as labels rather then as arguments and will report an error.

Functions, external routines, and the stack

The example program for this section implements a selection sort on an array of integers.

Listing 4. Implementation of selection sort on an integer array

Line NASM

```
001    section .data
002
003       array db
004          89, 10, 67, 1, 4, 27, 12, 34,
005             86, 3
006
007       ARRAY_SIZE equ $ - array
008
009
010       array_fmt db "  %d", 0
011
012
013       usort_str db "unsorted array:", 0
014
015
016       sort_str db "sorted array:", 0
017
018
019       newline db 10, 0
020
021
022
023    section .text
024       extern puts
025
026       global _start
027
028       _start:
029
030          push  usort_str
031          call  puts
032          add   esp, 4
033
034          push  ARRAY_SIZE
035          push  array
036          push  array_fmt
037          call  print_array10
038          add   esp, 12
039
040          push  ARRAY_SIZE
041          push  array
```

GAS

```
.section .data

   array:
      .byte  89, 10, 67, 1, 4, 27, 12,
             34, 86, 3

   array_end:
      .equ ARRAY_SIZE, array_end - array

   array_fmt:
      .asciz "  %d"

   usort_str:
      .asciz "unsorted array:"

   sort_str:
      .asciz "sorted array:"

   newline:
      .asciz "\n"


.section .text


   .globl _start

   _start:

      pushl $usort_str
      call  puts
      addl  $4, %esp

      pushl $ARRAY_SIZE
      pushl $array
      pushl $array_fmt
      call  print_array10
      addl  $12, %esp

      pushl $ARRAY_SIZE
      pushl $array
```

```
042         call  sort_routine20                          call  sort_routine20
043
044    ; Adjust the stack pointer                     # Adjust the stack pointer
045         add   esp, 8                                  addl  $8, %esp
046
047         push  sort_str                                pushl $sort_str
048         call  puts                                    call  puts
049         add   esp, 4                                  addl  $4, %esp
050
051         push  ARRAY_SIZE                              pushl $ARRAY_SIZE
052         push  array                                   pushl $array
053         push  array_fmt                               pushl $array_fmt
054         call  print_array10                           call  print_array10
055         add   esp, 12                                 addl  $12, %esp
056         jmp   _exit                                   jmp   _exit
057
058         extern printf
059
060      print_array10:                                 print_array10:
061         push  ebp                                     pushl %ebp
062         mov   ebp, esp                                movl  %esp, %ebp
063         sub   esp, 4                                  subl  $4, %esp
064         mov   edx, [ebp + 8]                          movl  8(%ebp), %edx
065         mov   ebx, [ebp + 12]                         movl  12(%ebp), %ebx
066         mov   ecx, [ebp + 16]                         movl  16(%ebp), %ecx
067
068         mov   esi, 0                                  movl  $0, %esi
069
070      push_loop:                                     push_loop:
071         mov   [ebp - 4], ecx                          movl  %ecx, -4(%ebp)
072         mov   edx, [ebp + 8]                          movl  8(%ebp), %edx
073         xor   eax, eax                                xorl  %eax, %eax
074         mov   al, byte [ebx + esi]                    movb  (%ebx, %esi, 1), %al
075         push  eax                                     pushl %eax
076         push  edx                                     pushl %edx
077
078         call  printf                                  call  printf
079         add   esp, 8                                  addl  $8, %esp
080         mov   ecx, [ebp - 4]                          movl  -4(%ebp), %ecx
081         inc   esi                                     incl  %esi
082         loop  push_loop                               loop  push_loop
083
084         push  newline                                 pushl $newline
085         call  printf                                  call  printf
086         add   esp, 4                                  addl  $4, %esp
087         mov   esp, ebp                                movl  %ebp, %esp
088         pop   ebp                                     popl  %ebp
089         ret                                           ret
090
091      sort_routine20:                                sort_routine20:
092         push  ebp                                     pushl %ebp
093         mov   ebp, esp                                movl  %esp, %ebp
094
095    ; Allocate a word of space in stack             # Allocate a word of space in stack
096         sub   esp, 4                                  subl  $4, %esp
097
098    ; Get the address of the array                  # Get the address of the array
099         mov   ebx, [ebp + 8]                          movl  8(%ebp), %ebx
100
101    ; Store array size                              # Store array size
102         mov   ecx, [ebp + 12]                         movl  12(%ebp), %ecx
103         dec   ecx                                     decl  %ecx
104
105    ; Prepare for outer loop here                   # Prepare for outer loop here
106         xor   esi, esi                                xorl  %esi, %esi
107
108      outer_loop:                                    outer_loop:
109    ; This stores the min index                     # This stores the min index
110         mov   [ebp - 4], esi                          movl  %esi, -4(%ebp)
111         mov   edi, esi                                movl  %esi, %edi
112         inc   edi                                     incl  %edi
113
114      inner_loop:                                    inner_loop:
115         cmp   edi, ARRAY_SIZE                         cmpl  $ARRAY_SIZE, %edi
```

```
116         jge   swap_vars
117         xor   al, al
118         mov   edx, [ebp - 4]
119         mov   al, byte [ebx + edx]
120         cmp   byte [ebx + edi], al
121         jge   check_next
122         mov   [ebp - 4], edi
123
124      check_next:
125         inc   edi
126         jmp   inner_loop
127
128      swap_vars:
129         mov   edi, [ebp - 4]
130         mov   dl, byte [ebx + edi]
131         mov   al, byte [ebx + esi]
132         mov   byte [ebx + esi], dl
133         mov   byte [ebx + edi], al
134
135         inc   esi
136         loop  outer_loop
137
138         mov   esp, ebp
139         pop   ebp
140         ret
141
142      _exit:
143         mov   eax, 1
144         mov   ebx, 0
145         int   80h
```

```
   jge   swap_vars
   xorb  %al, %al
   movl  -4(%ebp), %edx
   movb  (%ebx, %edx, 1), %al
   cmpb  %al, (%ebx, %edi, 1)
   jge   check_next
   movl  %edi, -4(%ebp)

check_next:
   incl  %edi
   jmp   inner_loop

swap_vars:
   movl  -4(%ebp), %edi
   movb  (%ebx, %edi, 1), %dl
   movb  (%ebx, %esi, 1), %al
   movb  %dl, (%ebx, %esi, 1)
   movb  %al, (%ebx,  %edi, 1)

   incl  %esi
   loop  outer_loop

   movl  %ebp, %esp
   popl  %ebp
   ret

_exit:
   movl  $1, %eax
   movl  $0, %ebx
   int   $0x80
```

Listing 4 might look overwhelming at first, but in fact it's very simple. The listing introduces the concept of functions, various memory addressing schemes, the stack and the use of a library function. The program sorts an array of 10 numbers and uses the external C library functions `puts` and `printf` to print out the entire contents of the unsorted and sorted array. For modularity and to introduce the concept of functions, the sort routine itself is implemented as a separate procedure along with the array print routine. Let's deal with them one by one.

After the data declarations, the program execution begins with a call to `puts` (line 31). The `puts` function displays a string on the console. Its only argument is the address of the string to be displayed, which is passed on to it by pushing the address of the string in the stack (line 30).

In NASM, any label that is not part of our program and needs to be resolved during link time must be predefined, which is the function of the `extern` keyword (line 24). GAS doesn't have such requirements. After this, the address of the string `usort_str` is pushed onto the stack (line 30). In NASM, a memory variable such as `usort_str` represents the address of the memory location itself, and thus a call such as `push usort_str` actually pushes the address on top of the stack. In GAS, on the other hand, the variable `usort_str` must be prefixed with `$`, so that it is treated as an immediate address. If it's not prefixed with `$`, the actual bytes represented by the memory variable are pushed onto the stack instead of the address.

Since pushing a variable essentially moves the stack pointer by a dword, the stack pointer is adjusted by adding 4 (the size of a dword) to it (line 32).

Three arguments are now pushed onto the stack, and the `print_array10` function is called (line 37). Functions are declared the same way in both NASM and GAS. They are nothing but labels, which are invoked using the `call` instruction.

After a function call, ESP represents the top of the stack. A value of `esp + 4` represents the return address, and a value of `esp + 8` represents the first argument to the function. All subsequent arguments are accessed by adding the size of a dword variable to the stack pointer (that is, `esp + 12`, `esp + 16`, and so on).

Once inside a function, a local stack frame is created by copying `esp` to `ebp` (line 62). You can also allocate space for local variables as is done in the program (line 63). You do this by subtracting the number of bytes required from `esp`. A value of `esp – 4` represents a space of 4 bytes allocated for a local variable, and this can continue as long as there is enough space in the stack to accommodate your local variables.

Listing 4 illustrates the base indirect addressing mode (line 64), so called because you start with a base address and add an offset to it to arrive at a final address. On the NASM side of the listing, `[ebp + 8]` is one such example, as is `[ebp – 4]` (line 71). In GAS, the addressing is a bit more terse: `4(%ebp)` and `-4(%ebp)`, respectively.

In the `print_array10` routine, you can see another kind of addressing mode being used after the `push_loop` label (line 74). The line is represented in NASM and GAS, respectively, like so:

NASM: `mov al, byte [ebx + esi]`

GAS: `movb (%ebx, %esi, 1), %al`

This addressing mode is the base indexed addressing mode. Here, there are three entities: one is the base address, the second is the index register, and the third is the multiplier. Because it's not possible to determine the number of bytes to be accessed from a memory location, a method is needed to find out the amount of memory addressed. NASM uses the byte operator to tell the assembler that a byte of data is to be moved. In GAS the same problem is solved by using a multiplier as well as using the `b`, `w`, or `l` suffix in the mnemonic (for example, `movb`). The syntax of GAS can seem somewhat complex when first encountered.

The general form of base indexed addressing in GAS is as follows:

```
%segment:ADDRESS (, index, multiplier)
```

or

```
%segment:(offset, index, multiplier)
```

or

```
%segment:ADDRESS(base, index, multiplier)
```

The final address is calculated using this formula:

```
ADDRESS or offset + base + index * multiplier.
```

Thus, to access a byte, a multiplier of 1 is used, for a word, 2, and for a dword, 4. Of course, NASM uses a simpler syntax. Thus, the above in NASM would be represented like so:

```
Segment:[ADDRESS or offset + index * multiplier]
```

A prefix of `byte`, `word`, or `dword` is used before this memory address to access 1, 2, or 4 bytes of memory, respectively.

Leftovers

Listing 5 reads a list of command line arguments, stores them in memory, and then prints them.

Listing 5. A program that reads command line arguments, stores them in memory, and prints them

Line	NASM	GAS
001	`section .data`	`.section .data`
002		
003	`; Command table to store at most`	`// Command table to store at most`
004	`;  10 command line arguments`	`//  10 command line arguments`
005	`   cmd_tbl:`	`   cmd_tbl:`
006	`      %rep 10`	`      .rept 10`
007	`         dd 0`	`         .long 0`
008	`      %endrep`	`      .endr`
009		
010	`section .text`	`.section .text`
011		
012	`   global _start`	`   .globl _start`
013		
014	`   _start:`	`   _start:`
015	`; Set up the stack frame`	`// Set up the stack frame`
016	`      mov   ebp, esp`	`      movl  %esp, %ebp`
017	`; Top of stack contains the`	`// Top of stack contains the`
018	`;  number of command line arguments.`	`//  number of command line arguments.`
019	`; The default value is 1`	`// The default value is 1`
020	`      mov   ecx, [ebp]`	`      movl  (%ebp), %ecx`
021		
022	`; Exit if arguments are more than 10`	`// Exit if arguments are more than 10`
023	`      cmp   ecx, 10`	`      cmpl  $10, %ecx`
024	`      jg    _exit`	`      jg    _exit`
025		
026	`      mov   esi, 1`	`      movl  $1, %esi`

```
027         mov   edi, 0
028
029    ; Store the command line arguments
030    ;  in the command table
031       store_loop:
032         mov   eax, [ebp + esi * 4]
033         mov   [cmd_tbl + edi * 4], eax
034         inc   esi
035         inc   edi
036         loop  store_loop
037
038         mov   ecx, edi
039         mov   esi, 0
040
041         extern puts
042
043       print_loop:
044    ; Make some local space
045         sub   esp, 4
046    ; puts function corrupts ecx
047         mov   [ebp - 4], ecx
048         mov   eax, [cmd_tbl + esi * 4]
049         push  eax
050         call  puts
051         add   esp, 4
052         mov   ecx, [ebp - 4]
053         inc   esi
054         loop  print_loop
055
056         jmp   _exit
057
058       _exit:
059         mov   eax, 1
060         mov   ebx, 0
061         int   80h
```

```
      movl  $0, %edi

// Store the command line arguments
//  in the command table
   store_loop:
      movl  (%ebp, %esi, 4), %eax
      movl  %eax, cmd_tbl( , %edi, 4)
      incl  %esi
      incl  %edi
      loop  store_loop

      movl  %edi, %ecx
      movl  $0, %esi


   print_loop:
// Make some local space
      subl  $4, %esp
// puts functions corrupts ecx
      movl  %ecx, -4(%ebp)
      movl  cmd_tbl( , %esi, 4), %eax
      pushl %eax
      call  puts
      addl  $4, %esp
      movl  -4(%ebp), %ecx
      incl  %esi
      loop  print_loop

      jmp   _exit

   _exit:
      movl  $1, %eax
      movl  $0, %ebx
      int   $0x80
```

Listing 5 shows a construct that repeats instructions in assembly. Naturally enough, it's called the repeat construct. In GAS, the repeat construct is started using the `.rept` directive (line 6). This directive has to be closed using an `.endr` directive (line 8). `.rept` is followed by a count in GAS that specifies the number of times the expression enclosed inside the `.rept/.endr` construct is to be repeated. Any instruction placed inside this construct is equivalent to writing that instruction `count` number of times, each on a separate line.

For example, for a count of 3:

```
.rept 3
     movl $2, %eax
.endr
```

This is equivalent to:

```
movl $2, %eax
movl $2, %eax
movl $2, %eax
```

In NASM, a similar construct is used at the preprocessor level. It begins with the `%rep` directive and ends with `%endrep`. The `%rep` directive is followed by an expression (unlike in GAS where the `.rept` directive is followed by a count):

```
%rep <expression>
     nop
%endrep
```

There is also an alternative in NASM, the `times` directive. Similar to `%rep`, it works at the assembler level, and it, too, is followed by an expression. For example, the above `%rep` construct is equivalent to this:

```
times <expression> nop
```

And this:

```
%rep 3
     mov eax, 2
%endrep
```

is equivalent to this:

```
times 3 mov eax, 2
```

and both are equivalent to this:

```
mov eax, 2
mov eax, 2
mov eax, 2
```

In Listing 5, the `.rept` (or `%rep`) directive is used to create a memory data area for 10 double words. The command line arguments are then accessed one by one from the stack and stored in the memory area until the command table gets full.

As for command line arguments, they are accessed similarly with both assemblers. ESP or the top of the stack stores the number of command line arguments supplied to a program, which is 1 by default (for no command line arguments). `esp + 4` stores the first command line argument, which is always the name of the program that was invoked from the command line. `esp + 8`, `esp + 12`, and so on store subsequent command line arguments.

Also watch the way the memory command table is being accessed on both sides in Listing 5. Here, memory indirect addressing mode (line 33) is used to access the command table along with an offset in ESI (and EDI) and a multiplier. Thus, `[cmd_tbl + esi * 4]` in NASM is equal to `cmd_tbl(, %esi, 4)` in GAS.

Conclusion

Even though the differences between these two assemblers are substantial, it's not that difficult to convert from one form to another. You might find that the AT&T syntax seems at first difficult to understand, but once mastered, it's as simple as the Intel syntax.

Resources

Learn

- Consult the NASM and GAS manuals for complete introductions to these two assemblers:
  - GAS: GNU Assembler
  - NASM: Netwide Assembler
- Read an explanation of selection sort on Wikipedia.
- In the developerWorks Linux zone, find more resources for Linux developers, and scan our most popular articles and tutorials.
- See all Linux tips and Linux tutorials on developerWorks.
- Stay current with developerWorks technical events and Webcasts.

Get products and technologies

- Order the SEK for Linux, a two-DVD set containing the latest IBM trial software for Linux from DB2®, Lotus®, Rational®, Tivoli®, and WebSphere®.
- With IBM trial software, available for download directly from developerWorks, build your next development project on Linux.

Discuss

- Get involved in the developerWorks community through blogs, forums, podcasts, and community topics in our new developerWorks spaces.

About the author

Ram holds a post graduate degree in computer science and is working as a software engineer in IBM's India Software Labs, Rational Division, developing and adding features to Rational ClearCase. He has worked on various flavors of Linux, UNIX, and Windows, as well as real-time mobile-based operating systems such as Symbian and Windows Mobile. In his spare time he hacks Linux and reads books.

Close [x]

developerWorks: Sign in

IBM ID:
Need an IBM ID?
Forgot your IBM ID?

Password:
Forgot your password?
Change your password

Keep me signed in.

By clicking **Submit**, you agree to the developerWorks terms of use.

Submit Cancel

The first time you sign into developerWorks, a profile is created for you. **Select information in your profile (name, country/region, and company) is displayed to the public and will accompany any content you post**. You may update your IBM account at any time.

All information submitted is secure.

Close [x]

Choose your display name

The first time you sign in to developerWorks, a profile is created for you, so you need to choose a display name. Your display name accompanies the content you post on developerWorks.

Please choose a display name between 3-31 characters. Your display name must be unique in the developerWorks community and should not be your email address for privacy reasons.

Display name: (Must be between 3 – 31 characters.)

By clicking **Submit**, you agree to the developerWorks terms of use.

Submit Cancel

All information submitted is secure.

1 star 1 star 1 star
2 stars 2 stars 2 stars
3 stars 3 stars 3 stars
4 stars 4 stars 4 stars
5 stars 5 stars 5 stars

Submit

Add comment:

Sign in or register to leave a comment.

Note: HTML elements are not supported within comments.

Notify me when a comment is added1000 characters left

Post

There is a problem in retrieving the comments. Please refresh the page later.

Print this page **Share this page** **Follow developerWorks**

About
Help
Contact us
Submit content

Feeds

Report abuse
Terms of use
Third party notice
IBM privacy
IBM accessibility

Faculty
Students
Business Partners